Lesson 1.2: Frontend engineering (React and Next.js)

What you learn

- **DOM manipulation.** The DOM is the live tree of page elements. Plain JavaScript changes it directly; React describes the page from state and updates the DOM for you.
- **Component lifecycle.** A component mounts, updates when its state or props change, and unmounts. In function components, useEffect runs side effects after render and its cleanup runs before the next effect or on unmount.
- **State management.** Keep state as local as possible. Share it with context for small cases, Zustand for light global stores, or Redux Toolkit when many parts of a large app need predictable, traceable updates.

Tools and resources (official sites, not checked live)

- React documentation
- Next.js documentation
- Zustand
- Redux Toolkit

ASSIGNMENT

Practice for lesson 1.2

Business problem: Build a counter and a shared cart count.

Steps

1. Create a Next.js app.
2. Build a counter with useState.
3. Move the count into a Zustand store and show it in two components.
4. Confirm both update together.

Expected output: Two components that stay in sync through one store.

SOLUTION (ANSWER KEY)

A store created with `create()` exposes the count and an increment function. Both components read it with the same hook, so one update re-renders both.

Check yourself

1. What does useEffect cleanup do?
   Answer: Runs before the next effect or on unmount to release timers or subscriptions.
2. When is Redux Toolkit worth it?
   Answer: Large apps with many interacting parts needing predictable, traceable state changes.

Lesson 1.2 of the **Development and IT** lessons. See this lesson and its official links on skillnavora.in · All chapters and lessons · Open the full lesson guide. Tools, prices and plans change; confirm on each official site.